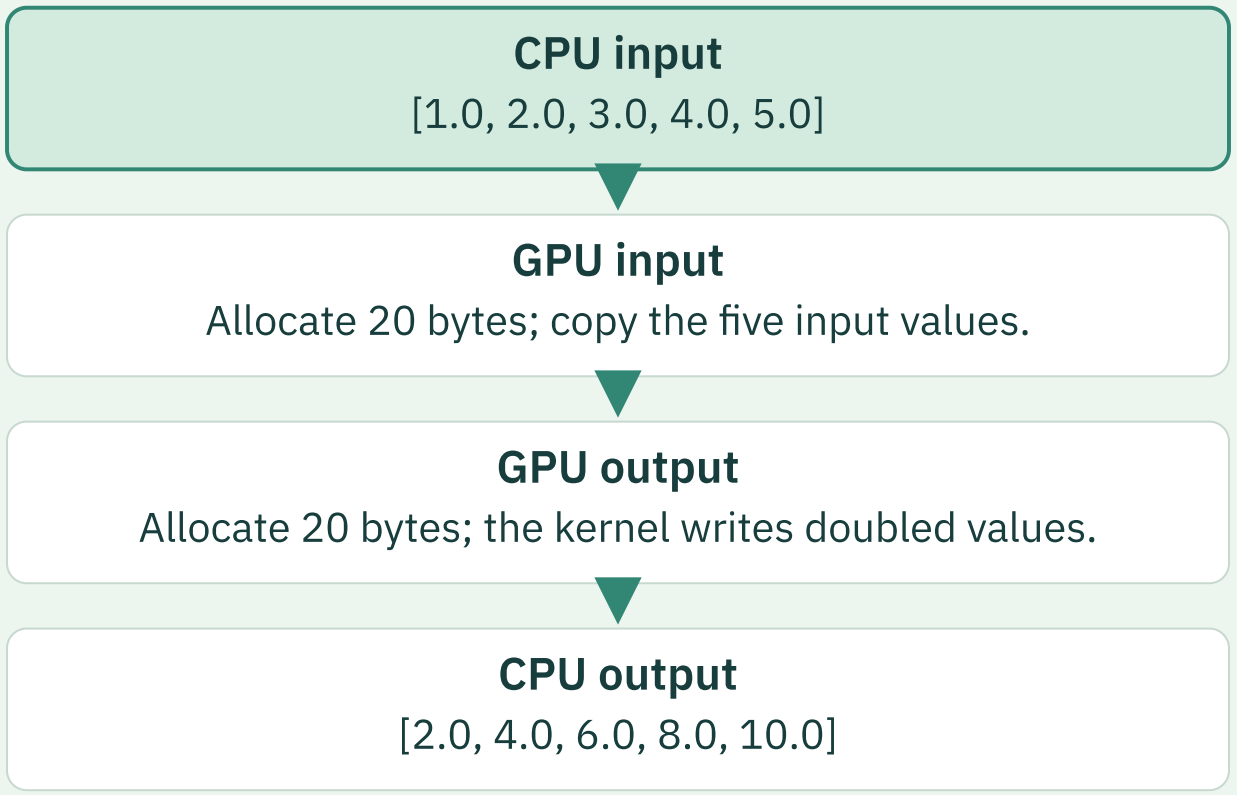
1 / 8

Start with the CPU input array.

The existing data supplies the element count.

> CPU-managed CUDA workflow · FP32 values · correctness, not timing



Allocation reserves storage; copying supplies values.

Copyright © 2026 Christopher Snow

2 / 8

Elements and bytes answer different questions.

Five FP32 values occupy twenty bytes.

> CPU-managed CUDA workflow · FP32 values · correctness, not timing

Count elements; allocate and copy bytes



5 elements × 4 bytes = 20 bytes

One FP32 array’s value storage

Input: 20 bytes · output: 20 bytes

Equal sizes here because doubling is one-to-one

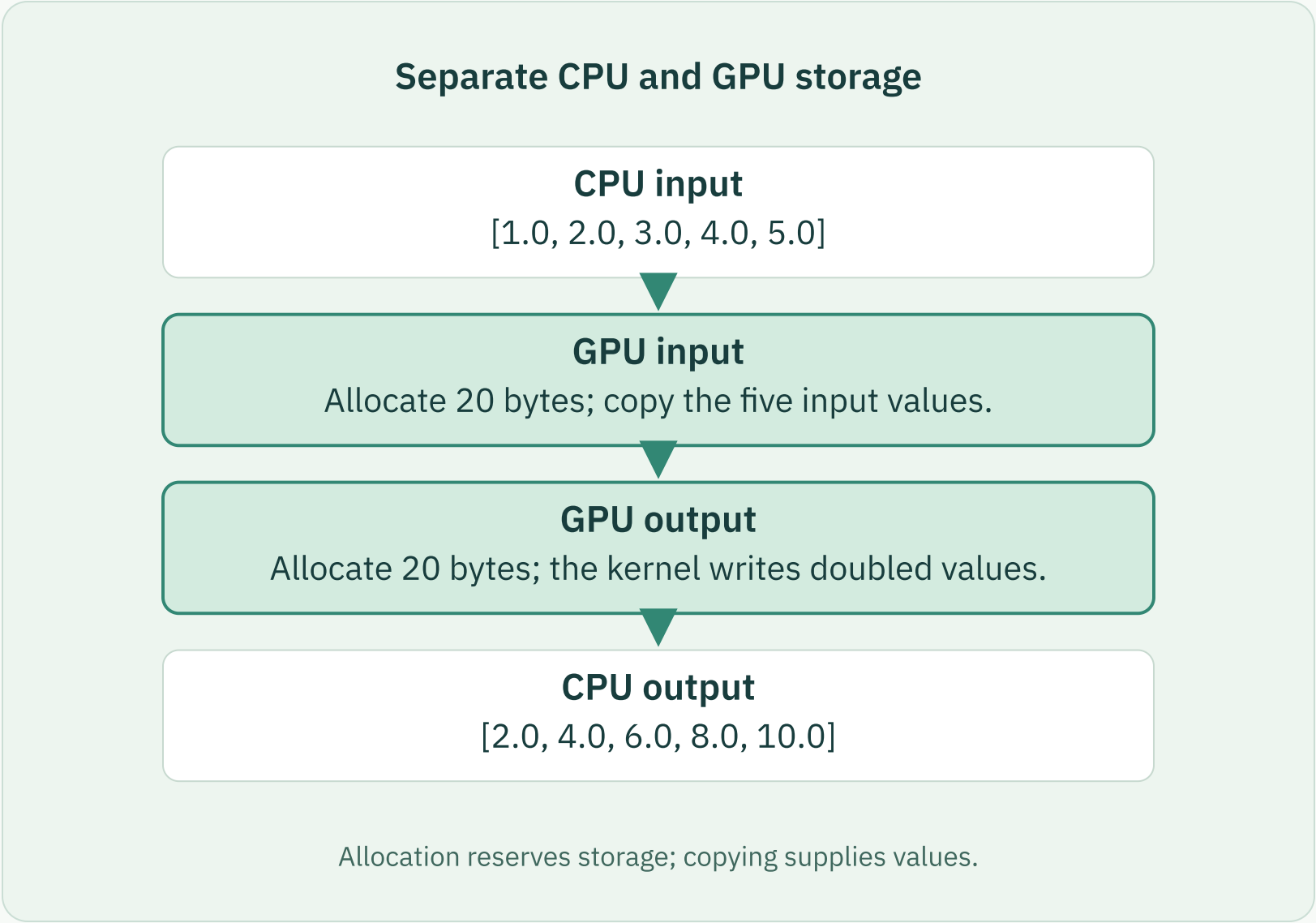
Copyright © 2026 Christopher Snow

3 / 8

Allocate GPU storage, then copy inputs.

Input storage and output storage are separate allocations.

> CPU-managed CUDA workflow · FP32 values · correctness, not timing



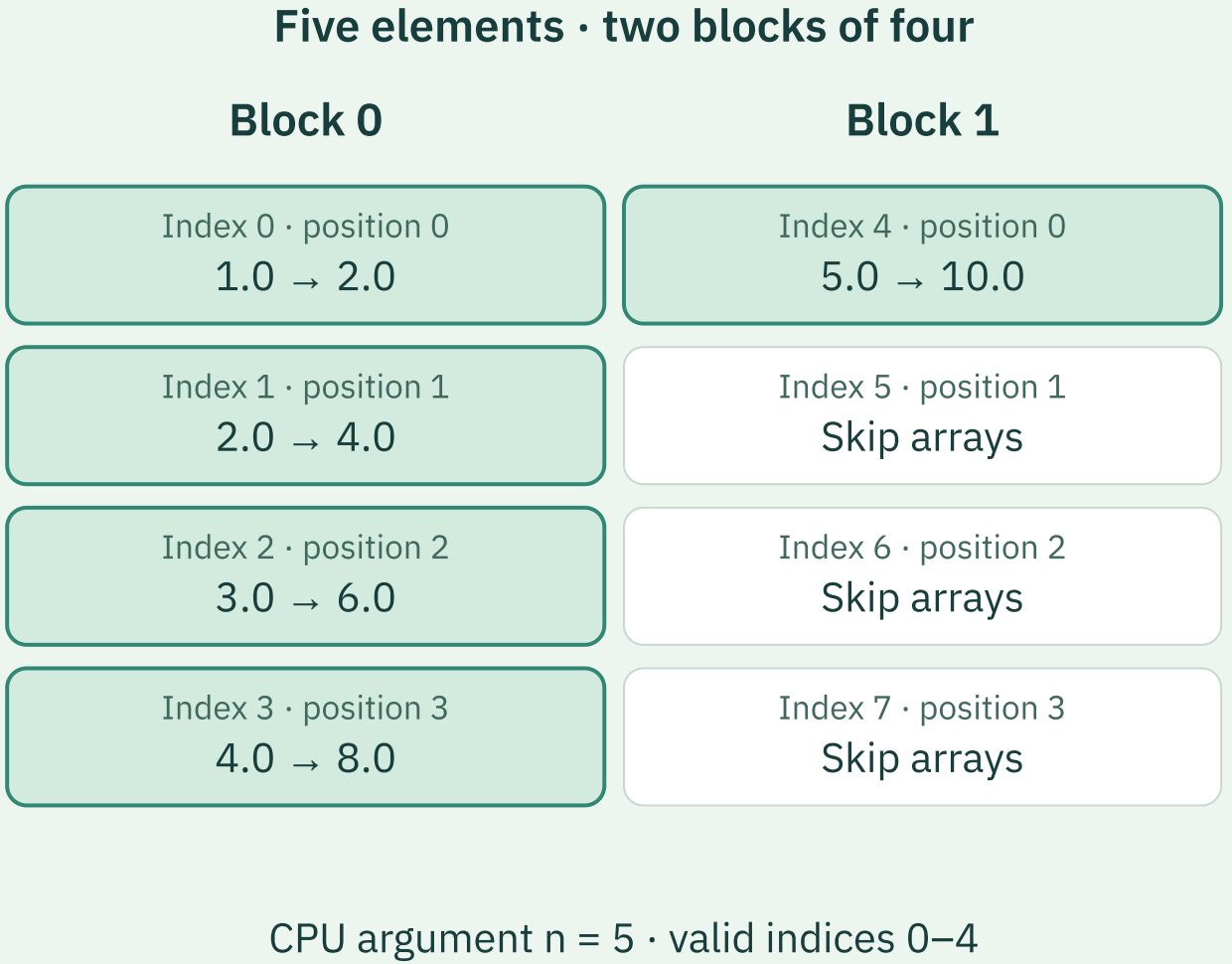
Copyright © 2026 Christopher Snow

4 / 8

Derive a launch that covers the data.

Two blocks of four cover five input positions.

> CPU-managed CUDA workflow · FP32 values · correctness, not timing



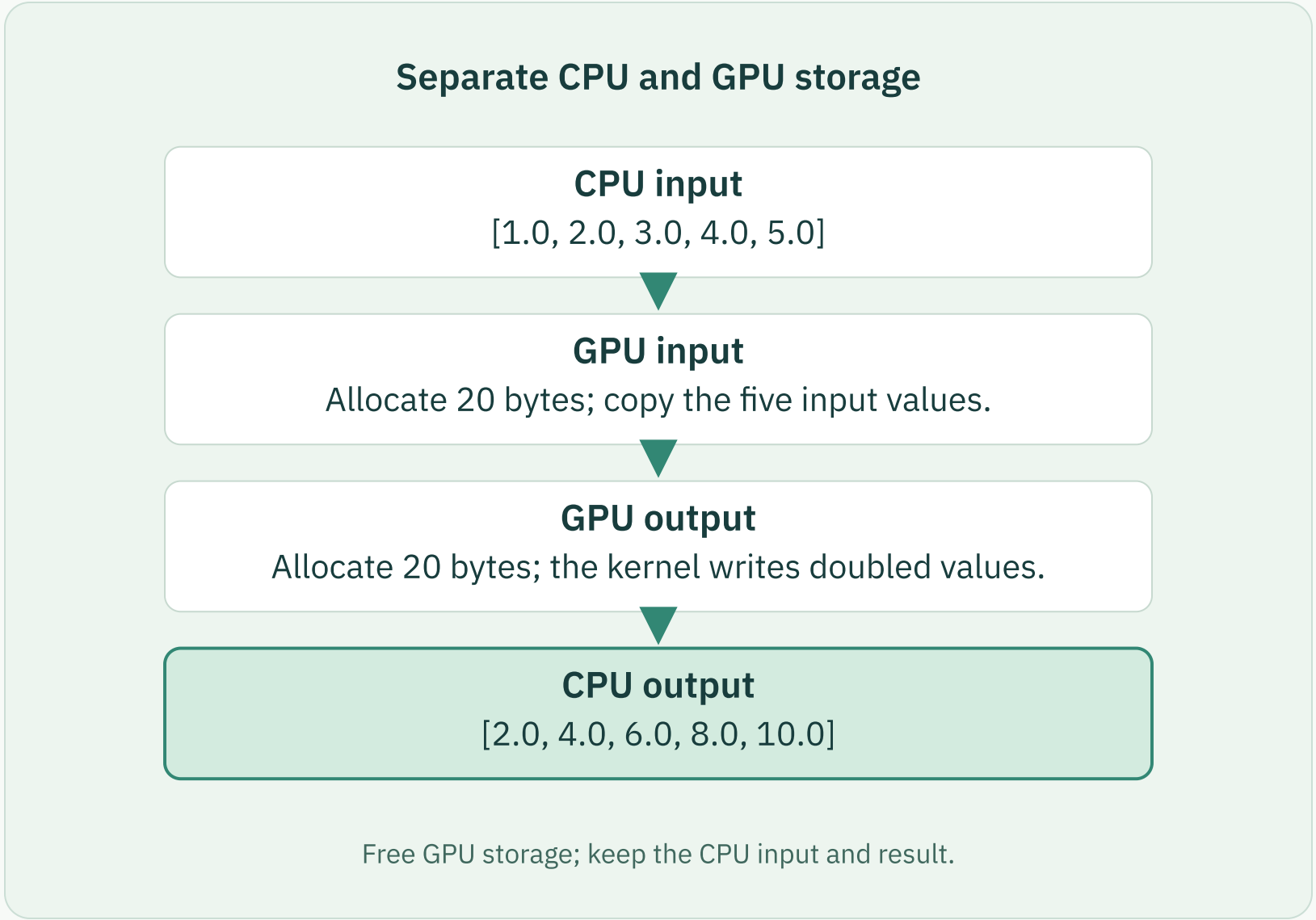
Copyright © 2026 Christopher Snow

5 / 8

Copy results back, then release GPU storage.

The CPU result remains after the GPU allocations are freed.

> CPU-managed CUDA workflow · FP32 values · correctness, not timing



Copyright © 2026 Christopher Snow

6 / 8

Pause: change the input array to nine values.

Keep FP32 values and four threads per block.

> CPU-managed CUDA workflow · FP32 values · correctness, not timing

CPU input vector: nine FP32 values

1.0	2.0	3.0	4.0
5.0	6.0	7.0	8.0
9.0			

Element count

?

Bytes per input/output array

?

Four-thread blocks

?

Copyright © 2026 Christopher Snow

7 / 8

Derive all three sizes from the input.

Nine elements need thirty-six bytes and three blocks.

> CPU-managed CUDA workflow · FP32 values · correctness, not timing

CPU input vector: nine FP32 values

1.0	2.0	3.0	4.0
5.0	6.0	7.0	8.0
9.0			

Element count
9

Bytes per input/output array
9 × 4 = 36 bytes

Four-thread blocks
3 blocks · 12 threads · skip 9, 10, 11

Copyright © 2026 Christopher Snow

8 / 8

Follow the complete ownership chain.

Input size → storage and copies → launch → result → cleanup.

> CPU-managed CUDA workflow · FP32 values · correctness, not timing

The CPU supplies the complete workflow

Input size and operation
Derive element count and required output shape.

Storage, copies and launch
Use bytes for storage/copies; elements for the guard.

Result and cleanup
Copy results to the CPU; then free GPU allocations.

A pointer carries an address, without an array length.

Copyright © 2026 Christopher Snow